

OPSFOLIO ECOSYSTEM

Developer Experience Tour

surveilr · Spry · Fleetfolio · Qualityfolio · Opsfolio Suite

Independent tools. Composable platform. Evidence by default.

surveilr

Spry

Fleetfolio

Qualityfolio

Opsfolio



THE OPSFOLIO ECOSYSTEM

Five products. Two infrastructure layers. One composable platform.

INFRASTRUCTURE LAYER



surveilr

Resource Surveillance & Integration Engine
RSSD · SQL Pages · Patterns · Single Binary



Spry

Markdown-as-Code · SQL Curator · DAG Orchestrator
BI as Code · Evidence Warehouses · Runbooks

APPLICATION LAYER — BUILT ON SURVEILR + SPRY

Fleetfolio

SQL-native EDR
Fleet Security

Qualityfolio

TMaC · Test Evidence
QA & Compliance

Opsfolio Suite

GRC Engineering
Operational Truth™

Each product can be used independently OR composed into the Opsfolio Suite

Why surveilr and Spry underpin everything



surveilr = Evidence Engine

Single static binary — no cluster, no runtime

Ingests any resource → portable SQLite RSSD

Every detection, event, test result → immutable SQL row

Built-in SQL Pages web server

AI-ready Operational Truth™ Warehouses



Spry = Orchestration Layer

Executable Markdown → DAG of SQL/Bash/TS tasks

Type-safe SQL libraries with [PI] + {attrs}

Builds SQLPage web apps from a single .md file

Evidence Warehouses — code + output in one artifact

BI as Code — dashboards version-controlled in Git



Together = Composable Platform

Spry orchestrates surveilr ingestion pipelines

surveilr stores evidence that Spry notebooks reference

Fleetfolio & Qualityfolio are surveilr Patterns + Spry views

Opsfolio Suite layers GRC policy on both

Each tool retains full independent DX

surveilr

Resource Surveillance & Integration Engine for Operational Truth™

Your resources. Your truth. Your control.



One Binary

Static. No cluster.
Runs anywhere.



RSSD

Portable SQLite
Operational Truth Warehouse



SQL Native

Standard SQL is
the only interface



AI-Ready

Deterministic substrate
for AI Workforce™



Evidence-First

Immutable, timestamped
by default

Oracle/PG/MySQL

Files + IMAP

REST APIs

HL7/FHIR/X.12

Capturable Execs

RSSD → Operational Truth™ Warehouse

SQL · JDBC/ODBC · MCPs · AI Skills · SQL Pages

Zero to Operational Truth™ in 3 commands

\$ Install surveilr

```
# macOS — Homebrew
brew tap surveilr/tap && brew install surveilr

# Linux — global install (Could use brew as well)
wget
https://github.com/surveilr/packages/releases/latest/download/
surveilr_jammy.deb
sudo dpkg -i surveilr_jammy.deb

# Manual — download + move binary
wget https://github.com/surveilr/packages/releases/\
download/latest/surveilr_x86_64-linux.tar.gz
tar -xzf surveilr_*.tar.gz && sudo mv surveilr /usr/local/bin/
surveilr --version
```

\$ Upgrade

```
# Always stay current
surveilr upgrade --yes
surveilr --version
```

\$ First run

```
# 1. Ingest files from the current directory
surveilr ingest files

# 2. Load a domain Pattern (e.g. Osquery)
deno run -A
https://github.com/surveilr/patterns/blob/main/osquery-
ms/queries.sql.ts | surveilr shell

# 3. Launch the built-in SQL Pages web UI
surveilr web-ui \
-d "resource-surveillance.sqlite.db" \
--port 3010 --host localhost

# Open → http://127.0.0.1:3010
```

- ✓ One binary — no daemon, no cluster, no JVM
- ✓ Runs on Linux · macOS · Windows · Raspberry Pi · Air-gap
- ✓ Data stays local — sovereign by default

Every resource lands in one normalized SQLite database

```
resource-surveillance.sqlite.db — core schema
-- Core ingestion tables (simplified)
CREATE TABLE uniform_resource (
  uniform_resource_id TEXT PRIMARY KEY,
  device_id TEXT NOT NULL,
  ingest_session_id TEXT NOT NULL,
  uri TEXT NOT NULL,          -- source path/URL
  content BLOB,               -- raw content
  nature TEXT,                -- file type / mime
  size_bytes INTEGER,
  last_modified_at TIMESTAMPTZ,
  content_digest TEXT,        -- SHA-256 hash
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE uniform_resource_csv ( -- type-specialized view
  uniform_resource_id TEXT PRIMARY KEY REFERENCES uniform_resource,
  header_names_csv_row TEXT,        -- detected columns
  parsed_data JSONB                -- structured rows
);
```

Portable

Copy one .db file
to share an entire warehouse

ACID + WAL

Safe concurrent reads,
no data loss on crash

Point-in-Time

Query data as it existed
at any moment T

Sync-Ready

Replicate to Postgres,
Oracle, MySQL at scale

The screenshot shows the SQLite editor interface with the 'uniform_resource' table selected. The table schema is displayed at the top, and a list of 10 rows of data is shown below. The columns are: uniform_resource_id, device_id, ingest_session_id, and device.

	uniform_resource_id TEXT NOT NULL PRIMARY KEY	device_id TEXT NOT NULL REFERENCES device(device_id) idx?	ingest_session_id TEXT NOT NULL REFERENCES ingest_session(ur_ingest_session_id) idx?	device
1	01KQYE4HP4M9GSM1NA3...	01KQYE4H00K0K2GZG55...	01KQYE4HP4V7M00K36450SHFK	device
2	01KQYE4HPQ3ASD3VKR6...	01KQYE4H00K0K2GZG55...	01KQYE4HP4V7M00K36450SHFK	device
3	01KQYE4HPSTQ2YFA35F...	01KQYE4H00K0K2GZG55...	01KQYE4HP4V7M00K36450SHFK	device
4	01KQYE4HPS78X5F9Z8G...	01KQYE4H00K0K2GZG55...	01KQYE4HP4V7M00K36450SHFK	device
5	01KQYE4HQ4ZK3KANQT...	01KQYE4H00K0K2GZG55...	01KQYE4HP4V7M00K36450SHFK	device
6	01KQYE4HQ56ZR88VJG3...	01KQYE4H00K0K2GZG55...	01KQYE4HP4V7M00K36450SHFK	device
7	01KQYE4HQ53BE96MBCJ...	01KQYE4H00K0K2GZG55...	01KQYE4HP4V7M00K36450SHFK	device
8	01KQYE4HWD53Y1W38ZF...	01KQYE4H00K0K2GZG55...	01KQYE4H43X82P3FWAHR8VG8MH	device
9	01KQYE4HVDQD341F359...	01KQYE4H00K0K2GZG55...	01KQYE4H43X82P3FWAHR8VG8MH	device
10	01KQYE4HWD56G9E19G7...	01KQYE4H00K0K2GZG55...	01KQYE4H43X82P3FWAHR8VG8MH	device

surveilr RSSD opened in SQLite editor— uniform_resource table

SQL is the only interface you need to know

Observability — stop searching logs, start querying

```
-- Contextual Unified Activity Records: all checkout failures
-- for premium users in the last hour with feature flag X
SELECT request_id, user_id, error_code, duration_ms, feature_flags
FROM activity_record
WHERE service      = 'checkout'
      AND status_code >= 400
      AND subscription = 'premium'
      AND time       > datetime('now', '-1 hour')
ORDER BY duration_ms DESC;
```

Security posture — live SQL against fleet state

```
-- Fleet posture: unencrypted devices
SELECT hostname, os_version, last_seen
FROM device_posture
WHERE disk_encrypted = FALSE
      AND last_seen > datetime('now', '-7 days');
```

Cross-system — joins across any ingested source

```
-- Cross-source correlation: deploys vs incidents
SELECT d.deploy_id, d.deployed_at, i.incident_title,
       i.opened_at, julianday(i.opened_at) - julianday(d.deployed_at) AS
lag_days
FROM   deploys d
JOIN   incidents i ON i.opened_at > d.deployed_at
ORDER BY lag_days ASC;
```

HCPCS	DESCRIPTION	CATEGORY	CONDITION	SOURCE	SERVICES	PATIENTS	MARKET VALUE	VAL/SERVICE
93306	The wickappler complete	Medicine & Monitoring	Heart Failure	CMS_GEO	20,223,412	18,034,566	\$2,316,445,065	\$114.54
93880	Extroconical biot study	Medicine & Monitoring	Heart Failure	CMS_GEO	5,565,562	4,765,239	\$649,236,275	\$125.89
93970	Extremity study	Medicine & Monitoring	Heart Failure	CMS_GEO	4,431,729	3,788,148	\$430,687,152	\$97.98
93229	Remote 30 day ecg tech supp	Medicine & Monitoring	Heart Failure	CMS_GEO	374,679	364,751	\$393,147,985	\$1,049.29
95490	Chronic care mgmt staff 1st 20	Evaluation & Management	Heart Failure	CMS_GEO	5,859,264	1,229,725	\$353,844,963	\$40.39
93925	Lower extremity study	Medicine & Monitoring	Heart Failure	CMS_GEO	1,933,611	1,646,331	\$391,126,700	\$175.39
93458	L.hrt artery/ventricle angio	Medicine & Monitoring	Heart Failure	CMS_GEO	1,190,598	1,128,594	\$322,206,450	\$270.63
93000	Electrocardiogram complete	Medicine & Monitoring	Heart Failure	CMS_GEO	20,539,330	13,743,592	\$298,074,455	\$14.51
93971	Extremity study	Medicine & Monitoring	Heart Failure	CMS_GEO	4,627,638	3,788,463	\$245,886,014	\$53.13

surveilr SQL Pages dashboard — query results rendered as interactive table with filters

```
surveilr shell --cmd "
SELECT json_group_array(
  json_object(
    'condition', condition_name,
    'hcpcs', hcpcs_code,
    'description', procedure_description,
    'beneficiaries', total_beneficiaries,
    'amount', total_allowed_amt
  )
) AS result
FROM condition_hcpcs_detail
LIMIT 5;
" | jq -r '[0].result | fromjson[]'
```

```
{
  "condition": "Type 2 Diabetes",
  "hcpcs": "95251",
  "description": "Cont gluc mntr analysis i&r",
  "beneficiaries": 318218,
  "amount": 22307959.5900029
}, {
  "condition": "Type 2 Diabetes",
  "hcpcs": "99213",
  "description": "Office o/p est low 20 min",
  "beneficiaries": 47821236,
  "amount": 12001466493.598
}, {
  "condition": "Type 2 Diabetes",
  "hcpcs": "99214",
  "description": "Office o/p est mod 30 min",
  "beneficiaries": 55057378,
  "amount": 23720517661.0936
}
```

surveilr CLI — surveilr shell --cmd 'SELECT ...' returning JSON output

Pre-packaged SQL solutions for specific domains

Each Pattern = stateless.sql (views) + stateful.sql (caches) + package.sql.ts (assembler). Install with one command.



Observability

Contextual Unified Activity Records. Replace text logs with SQL-queryable structured records.



Business Intelligence

Turn RSSD into a BI substrate. Connect Metabase, Superset, Power BI via ODBC/JDBC.



Data Quality

SQL-defined quality checks, freshness tests, and exception reporting against ingested resources.



DevOps & SRE

Capture deploys, incidents, on-call signals, and pipeline events into one queryable RSSD.



Financial Auditing

Aggregate transactional evidence for SOX-style audits with tamper-evident provenance.



Healthcare Compliance

HIPAA-aligned, HL7/FHIR-aware ingestion and evidence collection for regulated environments.



Message Queue

Durable SQL-native MQ with SQS-compatible facade. Queue history = audit evidence.



Security Monitoring

osquery-based endpoint config evidence, correlated for continuous security posture management.

Write SQL. Get a web app. No Node, no PHP, no Python stack.

 Qualityfolio

Total Test Case Count 28	Total Passed Cases 28	Total Failed Cases 0	Total Closed Cases 0
Test Failure Rate (Percentage) 0.0%	Test Success Rate (Percentage) 100.0%	Open Defects 0	Closed Defects 0
Test Cycle Plan (Month) 0	Unassigned Test Cases 0	Automation Test Coverage (%) 100.0%	Manual Test Coverage (%) 0%
Test Suite Count 1	Test Plan Count 1	Test Cycle Execution History	

SQL → rendered web table, no HTML needed

-- Every SELECT statement is a rendered UI component

```
SELECT 'table'          AS component,
       'Test Results' AS title;
```

```
SELECT test_case_id,
       exec_status  AS status,
       exec_nature  AS test_name,
       orch_started_at AS run_date
FROM   orchestration_session_exec
WHERE  exec_status = 'FAILED'
ORDER  BY orch_started_at DESC;
```

\$ Start web UI

```
# Launch in one command
surveilr web-ui \
  -d resource-surveillance.sqlite.db \
  --port 3010 \
  --host localhost
```

Open browser → <http://127.0.0.1:3010>

Dashboards

Evidence Portals

Admin Consoles

Compliance UIs

Internal Tools

Full developer experience. Your choice of scope.

surveilr — Standalone

- ✓ Data pipeline tool (ELT from any source → RSSD)
- ✓ Local-first observability (Contextual Unified Activity Records)
- ✓ SQL-native message queue without Kafka/RabbitMQ
- ✓ AI/RAG backend — structured Operational Truth as LLM context
- ✓ Portable evidence store for security tools (osquery, Trivy)
- ✓ One-command web dashboards via SQL Pages

surveilr inside Opsfolio Suite

- ➔ Powers the Evidence Warehouse — all Opsfolio controls store evidence here
- ➔ Fleetfolio reads fleet telemetry from surveilr RSSDs
- ➔ Qualityfolio orchestration sessions stored in surveilr tables
- ➔ Opsfolio GRC policy layer queries surveilr for control satisfaction
- ➔ Continuous audit posture = live SQL across surveilr evidence
- ➔ Framework-agnostic: SOC 2, CMMC, HIPAA, ISO 27001 from one RSSD

Spry

SQL Curator & Markdown-as-a-Programmable-Medium

Use Markdown to code, build, and orchestrate intelligence.

BI as Code

Dashboards, queries, metrics in Markdown.
Version and review like code.

Executable Runbooks

Ops procedures as Markdown.
Every step is a runnable cell.

SQL Libraries

Reusable type-safe SQL fragments
with [PI] + {attrs} binding.

SQL Migrations

Pure SQL + CTEs. Idempotent.
No migration framework required.

AI Context Eng.

Author prompts, embeddings, RAG
pipelines as Markdown cells.

Evidence Warehouses

Audit-ready notebooks: code
+ outputs for SOC2/HIPAA proof.

Every fenced code block is a runnable cell. Dependencies wire a DAG.

Spryfile.md — named tasks + deps + SQL routes

```
# Spryfile.md — a complete application in one Markdown file

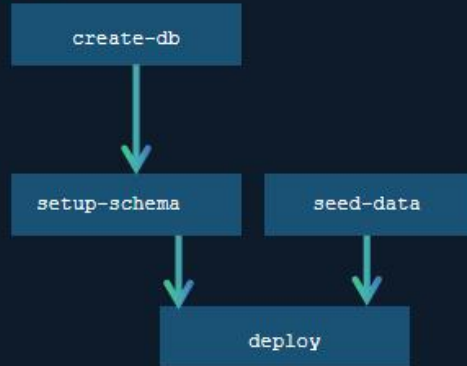
## setup
```bash create-db --descr "Create database file"
rm -f app.db && sqlite3 app.db "VACUUM;"
```
```bash setup-schema --dep create-db --descr "Apply schema"
sqlite3 app.db < schema.sql
```
```bash seed-data --dep setup-schema --descr "Load seed rows"
sqlite3 app.db < seed.sql
```
```bash deploy --dep setup-schema --dep seed-data
echo "Deployment completed"
```

## Routes

```sql index.sql { route: '/' }
SELECT 'hero' AS component,
 'My App' AS title,
 'Built with Spry' AS description;
```

```sql users.sql { route: '/users', cache: 60 }
SELECT 'table' AS component;
SELECT id, name, email FROM users;
```
```

TASK DAG



Key directives

| | |
|---------------------------------|--------------------------------------|
| <code>--descr "..."</code> | Human description of the task |
| <code>--dep <name></code> | Dependency on another named task |
| <code>{ route: '/' }</code> | SQL cell becomes a SQLPage URL route |
| <code>{ cache: 60 }</code> | Cache route response for N seconds |

Bootstrap to running app in seconds

\$ 1. Install Spry

```
# macOS / Linux — Homebrew (recommended)
brew install programmablemd/packages/spry

# Initialize a new Spry project
spry sp init

# Verify installation
spry --version
```

\$ 2. Develop & run

```
# Generate SQLPage filesystem + watch for changes
spry sp spc \
  --fs dev-src.auto \
  --conf sqlpage/sqlpage.json \
  --watch

# In another terminal — execute all tasks (DAG)
spry rb run

# Run a specific task + its deps
spry rb task deploy
```

Spry CLI reference — `spry <command>`

| | |
|--|---------------------------------------|
| <code>run</code> | Execute ALL tasks in dependency order |
| <code>task <name></code> | Run one task + its prerequisites |
| <code>spc --fs <dir></code> | Compile Spryfile → SQLPage filesystem |
| <code>spc --watch</code> | Live reload on Markdown changes (dev) |
| <code>spc --conf <file></code> | Specify SQLPage config (dev vs prod) |
| <code>--dialect sqlite postgres</code> | Choose database at init time |
| <code>--target-dir <path></code> | Set project directory at bootstrap |



Screenshot placeholder: [Spry CLI — spry rb run output showing task DAG execution with ✓ / ✗ per cell](#)

```
└─ spry rb run test.md --visualize ascii-tree
create-db
  └─ setup-schema
setup-schema
  ├── seed-data
  └─ deploy
seed-data
  └─ deploy
```

SQL cells become web routes. Notebooks become audit artifacts.

Spryfile.md — SQL cells wired to URL routes

```
```sql dashboard.sql { route: '/admin/dashboard', cache: 300 }
-- Named *.sql cell → SQLPage URL at /admin/dashboard
SELECT 'hero' AS component,
 'Control Posture' AS title;

SELECT 'table' AS component,
 'Active Gaps' AS title;
SELECT control_id, gap_type, detected_at, severity
FROM control_gaps
WHERE resolved_at IS NULL
ORDER BY severity DESC;
```
```

```
```sql evidence.sql { route: '/evidence' }
SELECT 'table' AS component, 'Evidence Log' AS title;
SELECT uri, content_digest, last_modified_at, nature
FROM uniform_resource
```

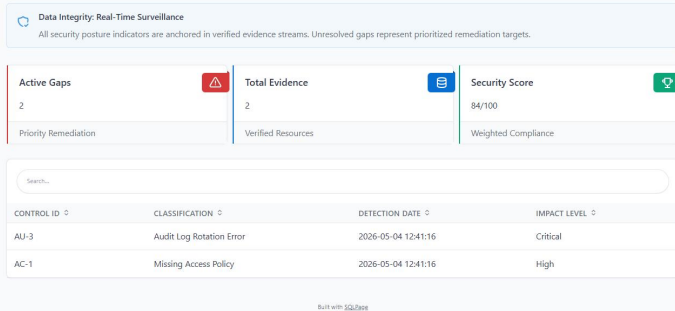
### Evidence Warehouse — executable quality checks as SQL cells (Soda.io alternative)

```
Evidence Warehouse — Data Quality as Code
Each check is a SQL cell that fails loudly on violation

```sql check-null-evidence --descr "No orphaned evidence"
SELECT CASE
  WHEN COUNT(*) > 0 THEN error('Orphaned evidence records found: ' || COUNT(*))
  ELSE 'PASS: all evidence linked to controls'
END
FROM uniform_resource ur
LEFT JOIN control_evidence ce ON ce.resource_id = ur.uniform_resource_id
WHERE ce.control_id IS NULL;
```

Enterprise Control Posture

Real-time synthesis of security control gaps and evidence integrity. Powered by the unified surveillance pipeline.



Evidence Warehouse

Comprehensive audit trail of ingested resources, content digests, and nature of evidence.

Search...

RESOURCE URI	CONTENT HASH	EVIDENCE NATURE	LAST VERIFIED
s3://logs/audit_log_2024.csv	sha256a1b2c3...	Log File	2026-05-06 12:41:16
https://internal/policy_v1.pdf	sha256d4e5f6...	Policy Document	2026-05-06 12:41:16

A complete developer tool on its own.

Use Case	Spry Standalone	Inside Opsfolio Suite
Data engineering pipelines	Mix SQL/Bash/TS in one Markdown. --dep wires DAG. Replaces dbt + Makefile.	Spry pipelines ingest + transform data that lands in surveilr RSSDs for Opsfolio controls.
Internal dashboards	Write SQL cells named *.sql. and spry sp spc serves them as a web app instantly.	Spry authors the SQLPage views used by Fleetfolio and Qualityfolio dashboards.
Runbooks / SRE	Executable Markdown procedures with real Bash cells. Single source of truth for ops.	Opsfolio uses Spry runbooks to encode and execute control testing procedures.
Data Quality checks	SQL cells as executable tests. Fail loudly. Re-run on every push. No YAML.	Control predicate checks ($\phi_i(E_t)$) are authored and run as Spry SQL cells.
AI context pipelines	Version prompts, embeddings, retrieval as cells. Reproducible RAG with full traceability.	Opsfolio AI Workforce™ agents are orchestrated by Spry for evidence collection tasks.
Spry has no dependency on surveilr. All features work standalone with any SQLite or PostgreSQL database.		

Fleetfolio

Device state you can query. Evidence you can prove.

HOW FLEETFOLIO USES SURVEILR

osquery agents

Steampipe

cnquery

SNMP polling

Cloud APIs (AWS/Azure/GCP)

Vuln scanners

surveilr RSSD → Fleetfolio SQL Views + SQL Pages

Detection & Response

SQL-native rules. Detection = evidence. Timestamped + auditable.

Asset Reconciliation

Expected vs Observed: MATCHED / MISSING / UNEXPECTED / RELOCATED

Posture & Policies

disk_encrypted, firewall, OS version. Drift alerts with before/after snapshots.

Threat Exposure Mgmt

Pen test orchestration. CVE correlation. POA&M generation.

Cloud Cost Discovery

Idle/oversized/zombie resources flagged. Savings in dollar amounts.

Compliance Mapping

SOC2, CMMC, FedRAMP, HIPAA, ISO 27001, NIST 800-53 out of box.

Detection rules are SQL. Evidence is SQL. Reconciliation is SQL.

Detection – Policy as SQL query

```
-- Policy: all laptops MUST have disk encryption
-- Detection fires as a row in device_posture_violations
SELECT hostname, disk_encrypted, os_version, last_seen
FROM   device_posture dp
JOIN   device_inventory di ON dp.device_id = di.device_id
WHERE  di.device_type    = 'laptop'
      AND dp.disk_encrypted = FALSE;
```

Drift Detection – before/after evidence snapshots

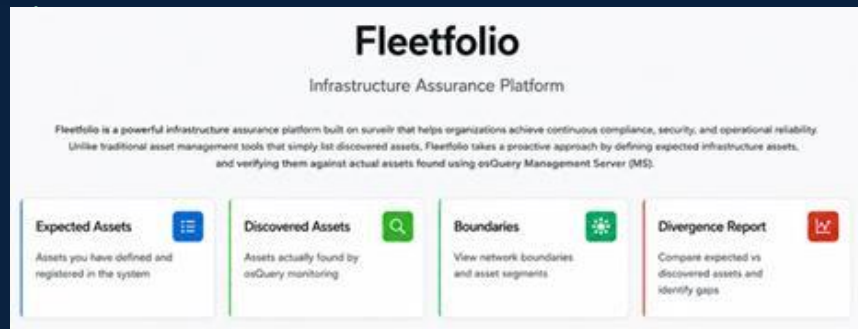
```
-- Drift detection: devices that WERE encrypted, now aren't
SELECT hostname, changed_at,
       old_value AS was_encrypted,
       new_value AS now_encrypted,
       evidence_before, evidence_after
FROM   device_posture_changes
WHERE  field      = 'disk_encrypted'
      AND old_value = TRUE
      AND new_value = FALSE
ORDER  BY changed_at DESC;
-- Each row has before/after evidence snapshots + audit trail
```

Evidence – complete incident package from one SQL query

```
-- Full incident evidence package (for any detection ID)
SELECT detection_id, detection_rule, detected_at,
       hostname, evidence_snapshot, remediation_status,
       remediation_completed_at, audit_chain
```

Asset Reconciliation – Expected vs Observed

```
-- Asset reconciliation: Expected vs Observed
-- Status: MATCHED | MISSING | UNEXPECTED | RELOCATED
SELECT COALESCE(e.asset_tag, d.serial_number) AS asset_id,
       e.expected_location, d.observed_location,
       CASE
         WHEN e.asset_tag IS NULL
         THEN 'UNEXPECTED'
         WHEN d.serial_number IS NULL
         THEN 'MISSING'
         WHEN e.expected_location != d.observed_location
         THEN 'RELOCATED'
         ELSE 'MATCHED'
       END AS reconciliation_status,
       d.last_seen
FROM   expected_assets e
FULL OUTER JOIN device_inventory d ON e.asset_tag =
```



 Fleetfolio SQL Pages – posture drift timeline: disk_encrypted TRUE → FALSE events over 30 days

Fleetfolio

Infrastructure Assurance Platform

Fleetfolio is a powerful infrastructure assurance platform built on surveil® that helps organizations achieve continuous compliance, security, and operational reliability. Unlike traditional asset management tools that simply list discovered assets, Fleetfolio takes a proactive approach by defining expected infrastructure assets, and verifying them against actual assets found using onQuery Management Server (MS).

Expected Assets



Assets you have defined and registered in the system

Discovered Assets



Assets actually found by onQuery monitoring

Boundaries



View network boundaries and asset segments

Divergence Report



Compare expected vs discovered assets and identify gaps

Qualityfolio

Evidence-Driven Quality Telemetry — Test Management as Code (TMaC)

TMaC ARTIFACT HIERARCHY

Project → Strategy → Suite → Plan → Cycle → Test Case → Evidence

TC-001.case.md — Markdown test case with YAML frontmatter

```
---
test_case_id: "TC-001"
title: "Verify MFA enforcement for PHI-bearing endpoints"
type: test_case
test_plan: TP-001
test_type: "Manual"
priority: "High"
tags: ["Security", "HIPAA", "SOC2-CC6.1"]
---

## Test Steps
1. Navigate to /api/phi-data without MFA token
2. Confirm HTTP 401 with MFA_REQUIRED error code
3. Complete MFA challenge and retry
4. Confirm HTTP 200 and access granted

## Expected Result
Access denied without MFA; granted after MFA challenge.
```

HOW QUALITYFOLIO USES SURVEILR

1

Author

Markdown + YAML test cases in Git. Full PR review workflow.

2

Ingest

surveilr ingest files — Markdown artifacts land in RSSD.

3

Execute

CLI/CI/manual run. Results captured as structured JSON evidence.

4

Store

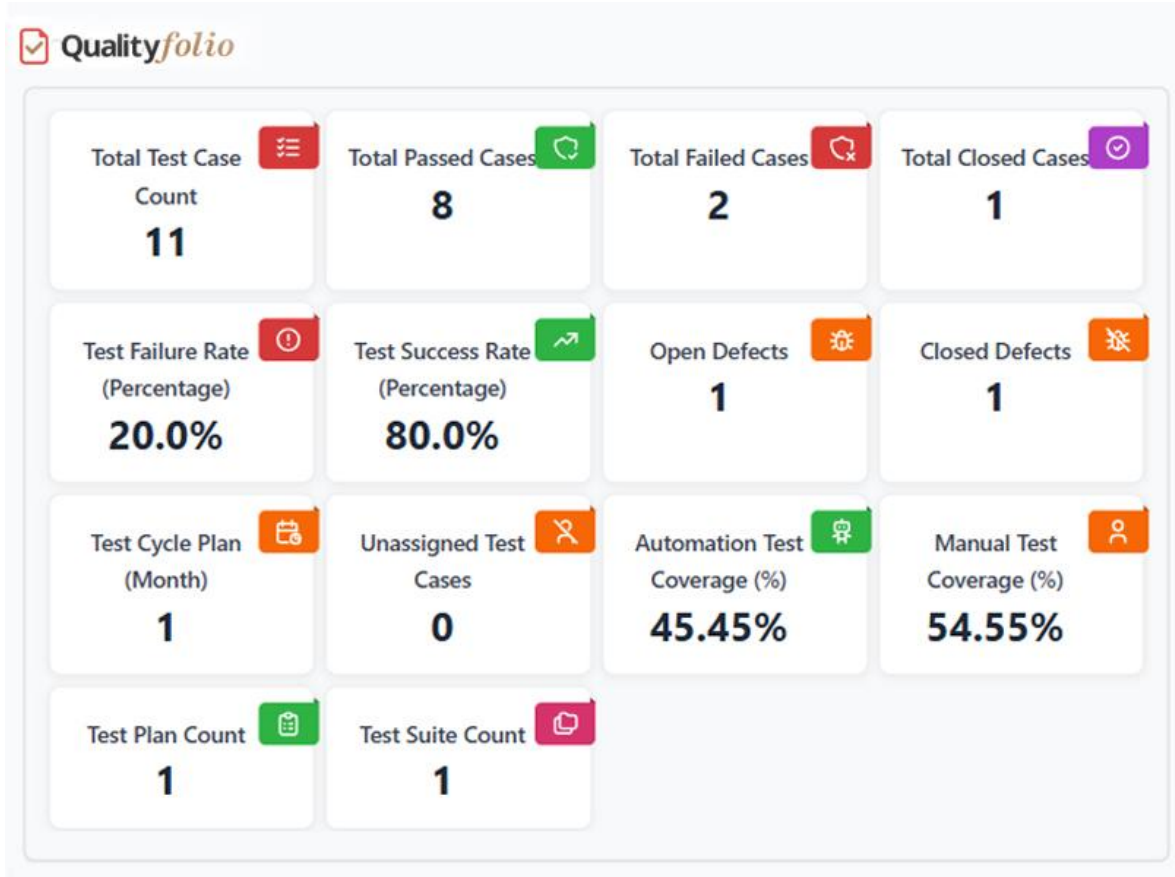
orchestration_session + orchestration_session_exec tables in RSSD.

5

Query

SQL evidence available instantly. Auditor self-service via SQL Pages.

 Qualityfolio web UI — [test cycle dashboard with pass/fail charts and evidence explorer](#)



Every test result is a queryable, immutable SQL row

Evidence schema – immutable, timestamped, auditable

```
-- surveilr orchestration tables (Qualityfolio's evidence
schema)
CREATE TABLE orchestration_session (
  orchestration_session_id TEXT PRIMARY KEY,
  device_id                TEXT NOT NULL,
  orchestration_nature_id  TEXT NOT NULL,
  version                  TEXT NOT NULL,
  orch_started_at          TIMESTAMPTZ NOT NULL,
  orch_finished_at         TIMESTAMPTZ,
  diagnostics_json         JSONB
);
CREATE TABLE orchestration_session_exec (
  orchestration_session_exec_id TEXT PRIMARY KEY,
  orchestration_session_id      TEXT REFERENCES
orchestration_session,
  exec_nature                    TEXT NOT NULL,    -- test name
  exec_status                    TEXT NOT NULL,    -- PASSED |
FAILED
  exec_code                      TEXT NOT NULL,    -- test body
```

\$ Install & run

```
# Install surveilr, run Qualityfolio, launch UI
spry rb run qualityfolio.md
spry sp spc --fs dev-src.auto --destroy-first --conf
sqlpage/sqlpage.json --md qualityfolio.md
EOH_INSTANCE=1 PORT=9227 surveilr web-ui -d ./resource-
surveillance.sqlite.db --port 9227 --host 0.0.0.0
```

SOC 2 evidence query – Q3 access controls

```
-- SOC 2: all access control tests in Q3 2025
SELECT ose.exec_nature AS test_name,
       ose.exec_status AS status,
       os.orch_started_at AS execution_time,
       ose.narrative_md AS evidence_summary
FROM   orchestration_session_exec ose
JOIN   orchestration_session os
      ON ose.orchestration_session_id =
os.orchestration_session_id
WHERE  ose.exec_nature LIKE '%access-control%'
      AND os.orch_started_at BETWEEN '2025-07-01' AND '2025-09-
30'
ORDER BY os.orch_started_at DESC;
```

Flaky test detection – assurance risk

```
-- Flaky tests – assurance risk report
SELECT ose.exec_nature AS test_name,
       COUNT(*) AS total_runs,
       SUM(CASE WHEN ose.exec_status='PASSED' THEN 1 ELSE 0
END) AS passes,
       ROUND(100.0 * SUM(CASE WHEN ose.exec_status='PASSED'
THEN 1 ELSE 0 END) / COUNT(*), 2)
AS pass_rate
FROM   orchestration_session_exec ose
GROUP BY ose.exec_nature
HAVING COUNT(*) > 5 AND pass_rate < 95
ORDER BY pass_rate ASC;
```



Details Evidences

Test Run Summary

Test Case: TC-QUALITYFOLIO-011

Status: FAILED

Description: Verify the design of Test Suite Detail page

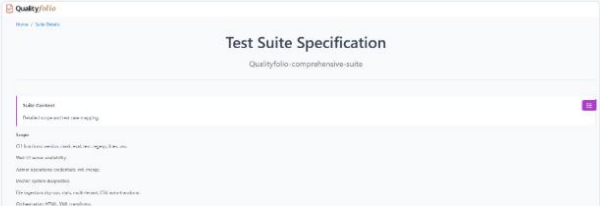
Duration: 300.00 seconds

Start Time: 2026-04-13T10:00:00.000Z

End Time: 2026-04-13T10:05:00.000Z

Cycle: 1.0.2

STEP #	STEP NAME	STATUS	START TIME	END TIME	ERROR
1	Log in to the Qualityfolio application with valid credentials.	PASSED	2026-04-13T10:00:00.000Z	2026-04-13T10:00:10.000Z	-
2	Navigate to the Test Suites overview page (e.g., /test-suite-cases-summary.sql).	PASSED	2026-04-13T10:00:10.000Z	2026-04-13T10:00:20.000Z	-
3	Click on a test suite to open the Test Suite Detail page.	PASSED	2026-04-13T10:00:20.000Z	2026-04-13T10:00:30.000Z	-
4	Inspect the page header for proper alignment, font size, and consistent styling of the suite name and metadata.	PASSED	2026-04-13T10:00:30.000Z	2026-04-13T10:01:00.000Z	-
5	Verify that all data tables render with uniform column widths, proper cell padding, and correct text alignment.	PASSED	2026-04-13T10:01:00.000Z	2026-04-13T10:01:30.000Z	-
6	Check that status badges (passed, failed, skipped) display the correct colors, labels, and consistent sizing.	PASSED	2026-04-13T10:01:30.000Z	2026-04-13T10:02:00.000Z	-
7	Verify that icons and visual indicators render without clipping, distortion, or misalignment.	FAILED	2026-04-13T10:02:00.000Z	2026-04-13T10:02:30.000Z	Some icons are clipped and misaligned on the Test Suite Detail page.
8	Review spacing and margins between sections for visual consistency and adherence to the design system.	SKIPPED			-
9	Resize the browser window to verify responsive layout behavior at common breakpoints (desktop, tablet, mobile).	SKIPPED			-
10	Check for any overlapping elements, text truncation, or broken UI components.	SKIPPED			-



Compliance is not a document state. It is a computational invariant.

OPERATIONAL TRUTH™ INVARIANT

$$\forall c_i \in C, \forall t : \phi_i(E_t) \vee g_i(t)$$

For all controls and all times — control is satisfied OR a gap signal is asserted. No unknown state.

surveilr

E_t — Evidence State

Ingests telemetry from all systems. Every RSSD row is a timestamped, immutable observed evidence point.

Spry

ϕ_i — Control Predicates

SQL cells express the expected control conditions. They're versioned, reviewed, and continuously evaluated.

Fleetfolio

Fleet Surface

Contributes device state, posture, and security telemetry as E_t evidence for infrastructure controls.

Qualityfolio

Quality Surface

Contributes test execution records and assurance evidence as E_t for software and process controls.

Every tool works independently. Compose as you grow.



Path 1 — surveilr Standalone

- 1 `brew install surveilr`
- 2 `surveilr ingest files`
- 3 `surveilr shell
pattern.sql`
- 4 `surveilr web-ui --port
3010`

Best for: data engineers, SRE, observability, compliance evidence



Path 2 — Spry Standalone

- 1 `brew install
programmablemd/packages/s
pry`
- 2 `Author Spryfile.md`
- 3 `spry rb run`
- 4 `spry sp spc --watch`

Best for: SQL pipelines, dashboards, runbooks, BI as Code



Path 3 — Fleetfolio / Qualityfolio

- 1 `Install surveilr`
- 2 `surveilr shell
qualityfolio/package.sql`
- 3 `surveilr ingest files`
- 4 `surveilr web-ui + open UI`

Best for: security teams, QA engineers, compliance officers



Path 4 — Full Opsfolio Suite

- 1 `Deploy surveilr + Spry
pipelines`
- 2 `Install Fleetfolio +
Qualityfolio patterns`
- 3 `Define controls as Spry
SQL cells`
- 4 `Query Operational Truth™
at any moment`

Best for: GRC engineering, continuous audit posture, AI Workforce™

GET STARTED TODAY

Independent tools. Composable platform. Evidence by default.

surveilr

[surveilr.com](#) · [docs.opsfolio.com/surveilr](#)

```
brew tap surveilr/tap && brew install surveilr
```

Spry

[sprymd.org](#) · [docs.opsfolio.com/spry](#)

```
brew install programmablemd/packages/spry
```

Fleetfolio

[fleetfolio.dev](#) · [docs.opsfolio.com/spry/fleetfolio/introduction](#)

```
sudo docker pull fleetfolio/fleetfolio && sudo docker run --rm -it --  
env-file .env -v $(pwd)/.env:/opt/aaa/.env -v  
$(pwd)/results:/opt/aaa/sessions/ fleetfolio/fleetfolio  
sudo docker pull fleetfolio/threat-exposure && sudo docker run --rm -it  
-p 9228:9228 -v "$PWD:/data" fleetfolio/threat-exposure
```

Qualityfolio

[qualityfolio.dev](#) · [next.qualityfolio.dev](#)

```
spry rb run qualityfolio.md  
spry sp spc --fs dev-src.auto --destroy-first --conf  
sqlpage/sqlpage.json --md qualityfolio.md  
EOH_INSTANCE=1 PORT=9227 surveilr web-ui -d ./resource-  
surveillance.sqlite.db --port 9227 --host 0.0.0.0
```

Opsfolio Suite · [opsfolio.com](#) · Compliance-as-a-Service — software + operating discipline delivered together